Understanding and Implementation of a Small-Scale Neural Radiance Field for Novel View Synthesis

Arpit Jain, Saransh Naik, Param Sonawane
Under the guidance of Dr. Pratik Shah
Department of Computer Science and Engineering, Indian Institute of Information Technology Vadodara, India
{202311016, 202311077, 202311082}@iiitvadodara.ac.in

***Abstract*—This report details the small-scale implementation of a Neural Radiance Field (NeRF) for novel view synthesis, based on the foundational 2020 paper by Mildenhall et al. The project's objective is to reconstruct a continuous volumetric scene representation from a sparse set of 2D images. The implemented system utilizes a multilayer perceptron (MLP) to map 3D spatial coordinates to volume density and color. This implementation, built in TensorFlow, was trained on a custom dataset of 60 images, each at 100×100 resolution. This paper documents the specific model architecture, which consists of an 8-layer MLP with 256-width hidden layers and a positional encoding dimension of $L = 10$. After training for 2000 iterations, the model achieved a final Peak Signal-to-Noise Ratio (PSNR) of 24.573, a Structural Similarity Index (SSIM) of 0.863, and a Learned Perceptual Image Patch Similarity (LPIPS) of 0.1644 on a held-out test view, demonstrating a high-fidelity reconstruction of the scene using limited computing power.**

***Index Terms*—Neural Radiance Fields, NeRF, View Synthesis, 3D Reconstruction, Volume Rendering, Deep Learning, Computer Vision.**

I. Introduction

The long-standing computer graphics challenge of novel view synthesis involves rendering photorealistic, previously unseen views of a 3D scene from a sparse collection of input 2D images and their corresponding camera poses. Achieving photorealism requires correctly modeling complex geometry and intricate, view-dependent material properties.

Prior to the work detailed in the paper by Mildenhall et al., leading methods for this task relied on scene representations that were often difficult to optimize or prohibitive in terms of memory. Approaches using discretized voxel grids, for example, suffer from poor time and space complexity, as scaling to higher resolutions requires a finer sampling of 3D space. Mesh-based representations, while memory-efficient, are often difficult to optimize via gradient descent from 2D images, as they require a fixed topology and can fall into local minima.

The introduction of Neural Radiance Fields (NeRF) provided a new paradigm that addresses these limitations. The core concept of NeRF is to represent a static scene as a continuous, 5D vector-valued function. This function, approximated by a deep, fully-connected network (or multilayer perceptron, MLP), maps a 5D coordinate—comprising a 3D spatial location $\mathbf{x} = (x, y, z)$ and a 2D viewing direction $\mathbf{d} = (\theta, \phi)$—to its corresponding volumetric density (σ) and view-dependent RGB color ($\mathbf{c}$).[1, 1]

To render an image from a novel viewpoint, camera rays are marched through this continuous representation. The output colors and densities sampled along the rays are then composited into a 2D image using classical volume rendering techniques. Because this entire process is differentiable, the weights of the MLP (Θ) can be optimized from scratch for a given scene using gradient descent. The only required input is a set of training images with known camera poses. The optimization minimizes the photometric error between the rendered images and the ground-truth images, encouraging the network to learn a coherent 3D model of the scene.

The primary objective of this report is to document a specific, end-to-end implementation of the NeRF framework, as realized in a Google Colab notebook.[1] This paper will meticulously detail the practical architecture, data processing pipeline, and training parameters used. A comprehensive quantitative and qualitative analysis of the experimental results is then presented to validate the implemented model's effectiveness in learning a high-fidelity 3D scene representation.

II. Implemented Methodology

This section details the specific, concrete implementation of the NeRF model as realized in the project's source code.[1] This implementation contains several key simplifications and deviations from the full model described in the original NeRF paper.

A. Neural Radiance Field Representation

The foundational NeRF paper describes an architecture that separates density (σ) as a function of location ($\mathbf{x}$) and color ($\mathbf{c}$) as a function of both location and viewing direction ($\mathbf{x}, \mathbf{d}$). This allows the model to learn view-dependent effects like specular reflections.

The model implemented in this project, however, utilizes a simplified, view-independent architecture. The '`init_model`' function defines an MLP whose input shape accounts only for the 3D position $\mathbf{x}$ and its positional encoding $\gamma(\mathbf{x})$.[1] The 2D viewing direction $\mathbf{d}$ is not used as an input. The network directly outputs a 4-dimensional vector corresponding to (R, G, B, σ). This implementation, therefore, models the scene as purely Lambertian, with $F_\Theta : (\mathbf{x}) \rightarrow (\mathbf{c}, \sigma)$.

The specific architecture is an 8-layer MLP with the following specifications [1]:

- **Input Layer:** A 63-dimensional vector, which is the 3D coordinate $\mathbf{x}$ concatenated with its positional encoding $\gamma(\mathbf{x})$.
- **Hidden Layers:** 8 fully-connected (Dense) layers, each with a width of $W = 256$ neurons and a ReLU activation function.
- **Skip Connection:** As defined in the '`init_model`' function, the original 63-dimensional input vector is concatenated with the output of the 4th hidden layer ('`if i%4==0 and i>0`').[1] This technique helps the network pass high-frequency spatial information to deeper layers.
- **Output Layer:** A final fully-connected layer with no activation function ('act=None') that maps the 256-channel feature vector to 4 output channels, representing the (R, G, B) color and volume density (σ).

B. Positional Encoding

Standard MLPs are known to be biased toward learning low-frequency functions, which, when applied to scene representations, results in overly-smoothed or blurry renderings that fail to capture fine detail. To address this, NeRF employs a positional encoding, mapping the input coordinates to a higher-dimensional space to enable the network to represent high-frequency variations in geometry and texture.[1, 1]

This project's 'posenc' function [1] implements this encoding. The function $\gamma(p)$ used for each coordinate component p is based on Eq. (6) in:

$$\gamma(p) = (\sin(2^0\pi p), \cos(2^0\pi p), \ldots, \sin(2^{L-1}\pi p), \cos(2^{L-1}\pi p))$$

The implementation sets the encoding parameter $L = 10$ (via '`L_embed = 10`').[1] The '`posenc`' function applies this to each of the three spatial coordinates (x, y, z) and also concatenates the original, un-encoded 3D coordinate. This results in the final 63-dimensional input vector for the MLP $(3 + 3 \times 2 \times 10 = 63)$.

C. Volume Rendering and Sampling Strategy

The color of a pixel is rendered by estimating the expected color $C(\mathbf{r})$ of the camera ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ that passes through it. This is computed using the volume rendering integral:

$$C(\mathbf{r}) = \int_{t_n}^{t_f} T(t)\sigma(\mathbf{r}(t))\mathbf{c}(\mathbf{r}(t))dt$$

where $T(t)$ is the accumulated transmittance from the near bound t_n to t:

$$T(t) = \exp\left(-\int_{t_n}^{t} \sigma(\mathbf{r}(s))ds\right)$$

This continuous integral is approximated numerically using a quadrature rule. The implementation uses the discretized formulation from:

$$\hat{C}(\mathbf{r}) = \sum_{i=1}^{N} T_i(1 - \exp(-\sigma_i\delta_i))\mathbf{c}_i$$

where $T_i = \exp(-\sum_{j=1}^{i-1} \sigma_j\delta_j)$ and $\delta_i = t_{i+1} - t_i$ is the distance between adjacent samples.[1, 1]

A key component of the original NeRF paper is stratified sampling, where the N samples are drawn at random from within N uniform bins along the ray. This is crucial for enabling the MLP to be optimized as a continuous function.

The '`render_rays`' function in this project's code [1] includes an '`if rand:`' conditional to perform stratified sampling. However, the training loop calls this function without setting '`rand=True`'.[1] Therefore, this implementation deviates from the paper and uses a deterministic, uniform sampling strategy. The $N = 64$ samples are taken at fixed, uniform intervals along the ray via '`tf.linspace`'. This may limit the model's ability to represent continuous geometry, as it is only ever queried at a fixed, discrete set of locations along each ray during training.

III. Experimental Setup

A. Dataset and Environment

The project was implemented in Google Colab, utilizing TensorFlow version 2.19.0.[1] The 'lpips' library (with the 'alex' network) and 'skimage.metrics.structural_similarity' were imported for quantitative model evaluation.[1]

The dataset is a custom scene loaded from the path '/content/my_dataset/28ecfec9d0854d0854d27a02866f870b8a7ba'.[1] The dataset consists of 100×100 pixel RGB images. Based on the metadata in 'transforms.json', the camera focal length was calculated to be 86.60.[1]

A total of 60 images and their corresponding camera poses were loaded. A train/test split was created by holding out the final image in the set ('images[-1]') as the test image. The first 59 images ('images[:-1]') were used for training the model.[1] The ground truth test image is shown in Figure 1.

B. Training Process

The model's weights (Θ) were optimized using the Adam optimizer [1] with a learning rate of 5×10^{-4}. The loss function is the total squared error (Mean Squared Error) between the rendered pixel colors ($\hat{C}(\mathbf{r})$) and the ground truth pixel colors ($C(\mathbf{r})$), as defined in [1, 1]:

$$\mathcal{L} = \sum_{r\in\mathcal{R}} ||\hat{C}(\mathbf{r}) - C(\mathbf{r})||_2^2$$

The model was trained for '`N_iters = 2000`' iterations.[1] For the volume rendering, the near and far bounds were set to '`NEAR_PLANE = 0.5`' and '`FAR_PLANE = 3.5`'.[1]

This implementation's training loop deviates from the "ray-batching" strategy described in [1], where a batch of rays is randomly sampled from the entire set of all pixels in all training images. Instead, this project uses an "image-batching" approach.[1] At each of the 2000 training iterations, a single image and its corresponding pose are randomly selected from the 59 training views. All $100 \times 100 = 10,000$ rays from this single image are used to compute the loss and update the model. This strategy is likely to produce a noisier gradient, as the model's update at any given step is biased towards a single viewpoint.

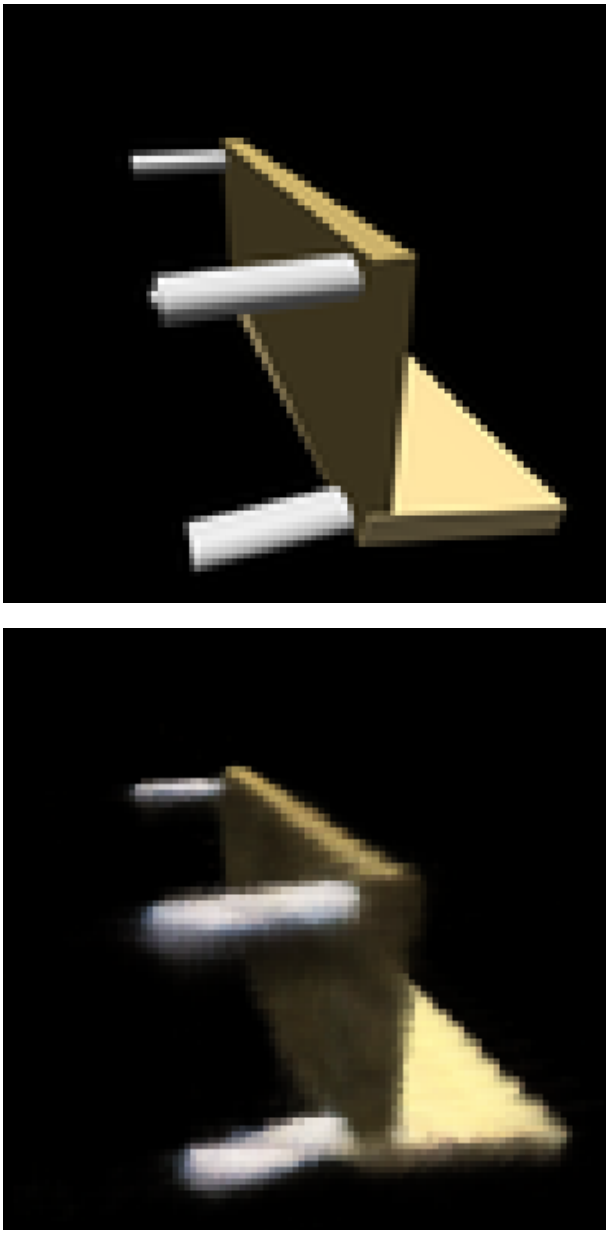

Fig. 1. Qualitative comparison of (a) the held-out ground truth test image [1] and (b) the final view rendered by the trained model at 2000 iterations.[1] The rendered image achieves a PSNR of 24.573 and SSIM of 0.863.

IV. Results and Analysis

This section presents the quantitative and qualitative results from training the model for 2000 iterations on the held-out test view.

A. Quantitative Analysis

The model's performance on the test view was evaluated at 25-iteration intervals using three standard metrics:

1) **PSNR (Peak Signal-to-Noise Ratio):** Measures the fidelity of the reconstruction. Higher is better.
2) **SSIM (Structural Similarity Index):** Measures the structural similarity between images, which correlates well with human perception.[1] Higher is better.
3) **LPIPS (Learned Perceptual Image Patch Similarity):** Measures perceptual similarity using a deep network ('alex' net), which is known to align well with human judgments of quality.[1] Lower is better.

The model's convergence and performance at key milestones are summarized in Table I. The model achieves a final PSNR of 24.573, an SSIM of 0.863, and an LPIPS of 0.1644. These strong results indicate a successful and high-fidelity reconstruction. The high SSIM, in particular, suggests the model has learned the scene's geometric structure very well.

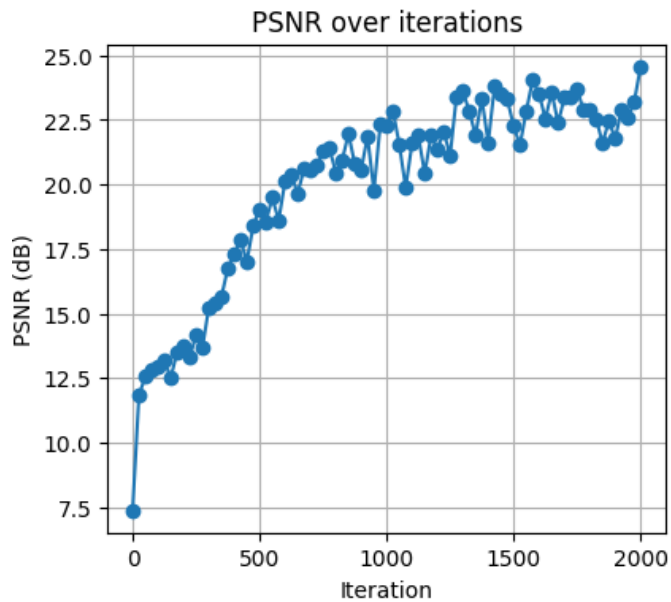

Fig. 2. PSNR (dB) on the held-out test view, plotted over 2000 training iterations. The plot shows rapid initial convergence followed by a noisy but steady improvement, consistent with the "image-batching" training strategy used.[1]

The convergence path, however, is not perfectly monotonic, which is visualized in Figure 2. For example, the PSNR at iteration 1500 (22.259 dB) is slightly lower than at iteration 1000 (22.310 dB). This "spiky" convergence is an expected consequence of the "image-batching" strategy, where the model's performance on the fixed test view fluctuates as it optimizes for different, individual training views at each step.

TABLE I
Quantitative Metric Progression on Test View

Iteration	PSNR (dB)	SSIM	LPIPS
0	7.367	0.029	1.0115
500	19.055	0.680	0.3322
1000	22.310	0.774	0.2275
1500	22.259	0.823	0.1715
2000	**24.573**	**0.863**	**0.1644**

Source: Data extracted from Colab notebook output logs.[1]

B. Qualitative Analysis

A qualitative comparison of the ground truth test image and the final rendered view from the trained model is presented in Fig. 1. The rendered image (b) is a high-fidelity reconstruction of the ground truth (a). The overall geometry, color, and structure are accurately captured. Despite the model's view-independent simplification, it has clearly learned a coherent and detailed 3D representation.

Figure 2 plots the PSNR on the test view over all 2000 training iterations. This graph visually confirms the analysis from Table I, showing a rapid initial learning phase in the first $\sim$500 iterations, followed by a noisier, "spiky" convergence to the final, optimal PSNR.

Finally, the trained model was successfully used to generate a smooth 360-degree video path. This was achieved by rendering a sequence of 120 novel views from interpolated spherical poses.[1] This successfully demonstrates that the model has learned a continuous representation and fulfilled the core project goal of novel view synthesis.

V. Conclusion

This report has documented the implementation, training, and analysis of a Neural Radiance Field. The project, built in TensorFlow, utilized a simplified 8-layer MLP with $L = 10$ positional encoding. This model differed from the original NeRF paper by being view-independent (Lambertian) and by using deterministic, uniform ray sampling.

The model was trained for 2000 iterations on a 100×100 60-image dataset, using an "image-batching" strategy. This process yielded a strong final performance on a held-out test view, achieving a PSNR of 24.573, SSIM of 0.863, and LPIPS of 0.1644.[1]

The quantitative and qualitative results, including the successful generation of smooth videos from novel viewpoints [1], confirm that this implementation effectively replicates the core principles of NeRF. It validates the method's ability to learn a coherent, continuous 3D scene representation from a sparse set of 2D images, even with a simplified model architecture and modified training procedure.

References

[1] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis," *Communications of the ACM*, vol. 65, no. 1, pp. 99–106, Jan. 2022, DOI: 10.1145/3503250.